

Graph Algorithms: Minimum Spanning Tree and Topological Sort

Yashdeep Singh

ANCC IIT Delhi

- We will discuss the **Minimum Spanning Tree (MST)** problem.
- We will cover two greedy MST algorithms: **Prim's algorithm** and **Kruskal's algorithm**.
- We will see why **DSU / Union-Find** is used in Kruskal.
- We will then discuss **topological sorting** of DAGs.
- The focus is on intuition, correctness, and implementation.

Minimum Spanning Tree: problem statement

Input

A connected, weighted, undirected graph $G = (V, E)$ with $|V| = n$ and edge weights $w(e)$.

Minimum Spanning Tree: problem statement

Input

A connected, weighted, undirected graph $G = (V, E)$ with $|V| = n$ and edge weights $w(e)$.

Goal

Find a spanning tree with minimum total edge weight.

Minimum Spanning Tree: problem statement

Input

A connected, weighted, undirected graph $G = (V, E)$ with $|V| = n$ and edge weights $w(e)$.

Goal

Find a spanning tree with minimum total edge weight.

A spanning tree:

- connects all vertices,
- has exactly $n - 1$ edges,
- contains no cycle.



Minimum Spanning Tree: problem statement

Input

A connected, weighted, undirected graph $G = (V, E)$ with $|V| = n$ and edge weights $w(e)$.

Goal

Find a spanning tree with minimum total edge weight.

A spanning tree:

- connects all vertices,
- has exactly $n - 1$ edges,
- contains no cycle.

Important

If the graph is disconnected, then an MST does not exist.

Prim's algorithm: intuition

Start from any vertex. Repeatedly attach the minimum-weight edge that connects the current tree to a new vertex.

Prim's algorithm: intuition

Start from any vertex. Repeatedly attach the minimum-weight edge that connects the current tree to a new vertex.

- 1 Choose an arbitrary starting vertex.
- 2 Maintain the set of vertices already in the tree.
- 3 Among all edges leaving this set, pick the cheapest one.
- 4 Add the new vertex and continue.

Prim's algorithm: intuition

Start from any vertex. Repeatedly attach the minimum-weight edge that connects the current tree to a new vertex.

- 1 Choose an arbitrary starting vertex.
- 2 Maintain the set of vertices already in the tree.
- 3 Among all edges leaving this set, pick the cheapest one.
- 4 Add the new vertex and continue.

At every step, Prim only cares about the boundary between the built tree and the remaining graph.

Prim's algorithm: implementation options

- For every unselected vertex, store the cheapest known edge to the tree.
- Dense version: keep this information in arrays and scan linearly.
- Sparse version: keep it in a heap or set and update by adjacency list.

Prim's algorithm: implementation options

- For every unselected vertex, store the cheapest known edge to the tree.
- ✓• Dense version: keep this information in arrays and scan linearly.
- ✓• Sparse version: keep it in a heap or set and update by adjacency list.

- Dense graphs: $O(n^2)$ with an adjacency matrix or full scan.
- Sparse graphs: $O(m \log n)$ with a heap and adjacency list.

Prim's algorithm: dense graph implementation

```
✓vector<bool> used(n, false);  
vector<int> min_e(n, INF), parent(n, -1);  
min_e[0] = 0;  
  
✓for (int i = 0; i < n; ++i) {  
    int ✓v = -1;  
    for (int j = 0; j < n; ++j)  
        if (✓!used[j] && (v == -1 || min_e[j] < min_e[v]))  
            v = j;  
  
    used[v] = true;  
    total += min_e[v];  
  
    for (int to = 0; to < n; ++to) {  
        if (adj[v][to] < min_e[to]) {  
            min_e[to] = adj[v][to];  
            parent[to] = v;  
        }  
    }  
}
```

Prim's algorithm: dense graph implementation

```
vector<bool> used(n, false);
vector<int> min_e(n, INF), parent(n, -1);
min_e[0] = 0;

for (int i = 0; i < n; ++i) {
    int v = -1;
    for (int j = 0; j < n; ++j)
        if (!used[j] && (v == -1 || min_e[j] < min_e[v]))
            v = j;

    used[v] = true;
    total += min_e[v];

    for (int to = 0; to < n; ++to) {
        if (adj[v][to] < min_e[to]) {
            min_e[to] = adj[v][to];
            parent[to] = v;
        }
    }
}
```

- Best for dense graphs.

Prim's algorithm: sparse graph implementation

```
while (!pq.empty()) {  
    auto [w, v] = pq.top();  
    pq.pop();  
    if (used[v]) continue;  
    used[v] = true;  
    total += w;  
  
    for (auto [to, wt] : adj[v]) {  
        if (!used[to]) {  
            pq.push({wt, to});  
        }  
    }  
}
```



Prim's algorithm: sparse graph implementation

```
while (!pq.empty()) {  
    auto [w, v] = pq.top();  
    pq.pop();  
    if (used[v]) continue;  
    used[v] = true;  
    total += w;  
  
    for (auto [to, wt] : adj[v]) {  
        if (!used[to]) {  
            pq.push({wt, to});  
        }  
    }  
}
```

- The queue may contain stale entries.
- The `used[]` array skips vertices already added to the tree.
- Best for sparse graphs.

Prim's algorithm: why it works

Suppose Prim chooses an edge e that is not in some fixed MST. In that MST, the path between the same two components must contain another edge f crossing the same cut. Since Prim always chooses the lightest crossing edge, $w(e) \leq w(f)$.

Prim's algorithm: why it works

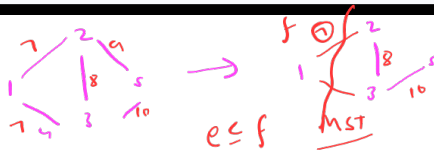
Suppose Prim chooses an edge e that is not in some fixed MST. In that MST, the path between the same two components must contain another edge f crossing the same cut. Since Prim always chooses the lightest crossing edge, $w(e) \leq w(f)$.

- Replace f by e .
- The graph stays connected and cycle-free.
- The total weight does not increase.

Prim's algorithm: why it works

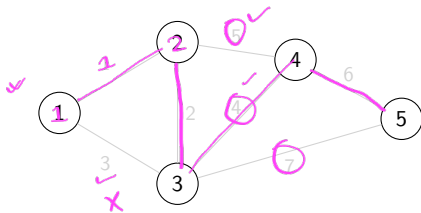
Suppose Prim chooses an edge e that is not in some fixed MST. In that MST, the path between the same two components must contain another edge f crossing the same cut. Since Prim always chooses the lightest crossing edge, $w(e) \leq w(f)$.

- Replace f by e .
- The graph stays connected and cycle-free.
- The total weight does not increase.



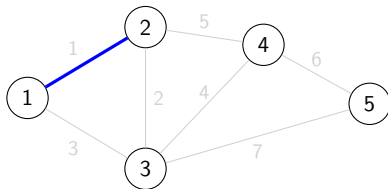
Every greedy step can be made compatible with an MST, so the final tree is optimal.

Prim example



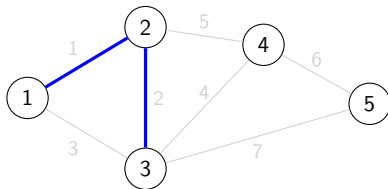
- Start from vertex 1.

Prim example



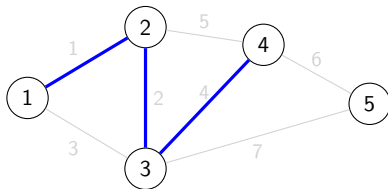
- Start from vertex 1.
- Pick (1, 2).

Prim example



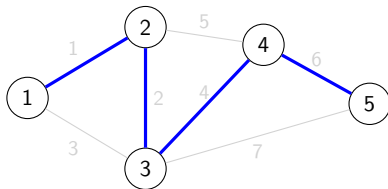
- Start from vertex 1.
- Pick (1, 2).
- Then pick (2, 3).

Prim example



- Start from vertex 1.
- Pick (1, 2).
- Then pick (2, 3).
- Then pick (3, 4).

Prim example



- Start from vertex 1.
- Pick (1, 2).
- Then pick (2, 3).
- Then pick (3, 4).
- Finally pick (4, 5).

Kruskal's algorithm: intuition

Sort all edges by weight. Add an edge if and only if it connects two different components.

Kruskal's algorithm: intuition

Sort all edges by weight. Add an edge if and only if it connects two different components.

- 1 Start with n isolated vertices.
- 2 Process edges in nondecreasing order.
- 3 Skip an edge if it would create a cycle.
- 4 Stop after taking $n - 1$ edges.

Kruskal's algorithm: intuition

Sort all edges by weight. Add an edge if and only if it connects two different components.

- 1 Start with n isolated vertices.
- 2 Process edges in nondecreasing order.
- 3 Skip an edge if it would create a cycle.
- 4 Stop after taking $n - 1$ edges.

Instead of growing one tree, Kruskal grows many components and merges them.

Kruskal's algorithm: step-by-step view

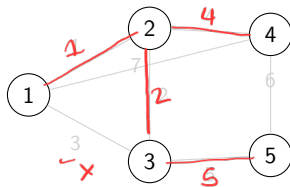
- The only thing we need to know is whether two endpoints are already connected.
- If they are in different components, the edge is safe to add.
- If they are in the same component, adding the edge would create a cycle.

Kruskal's algorithm: step-by-step view

- The only thing we need to know is whether two endpoints are already connected.
- If they are in different components, the edge is safe to add.
- If they are in the same component, adding the edge would create a cycle.

In any cycle, the heaviest edge is never needed in an MST if a lighter alternative already connects the same vertices.

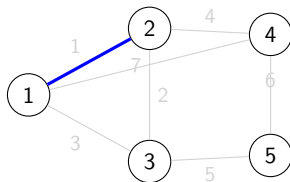
Kruskal example



- Sort edges by weight.



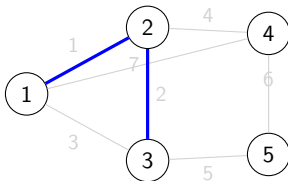
Kruskal example



- Sort edges by weight.
- Take (1, 2).

Kruskal always adds the lightest edge that joins two different components.

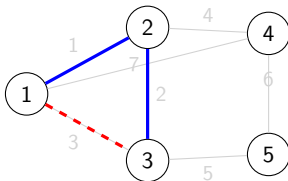
Kruskal example



- Sort edges by weight.
- Take (1, 2).
- Take (2, 3).

Kruskal always adds the lightest edge that joins two different components.

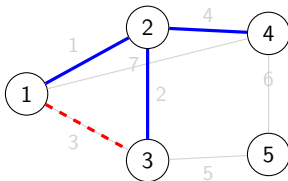
Kruskal example



- Sort edges by weight.
- Take (1, 2).
- Take (2, 3).
- Skip (1, 3) because it forms a cycle.

Kruskal always adds the lightest edge that joins two different components.

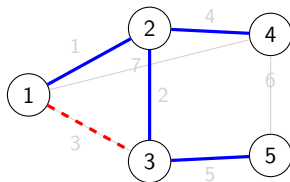
Kruskal example



- Sort edges by weight.
- Take (1, 2).
- Take (2, 3).
- Skip (1, 3) because it forms a cycle.
- Take (2, 4).

Kruskal always adds the lightest edge that joins two different components.

Kruskal example



- Sort edges by weight.
- Take (1, 2).
- Take (2, 3).
- Skip (1, 3) because it forms a cycle.
- Take (2, 4).
- Take (3, 5) and stop.

Kruskal always adds the lightest edge that joins two different components.

Disjoint Set Union (DSU)

What is DSU

A partition of vertices into disjoint sets, with operations:

- `make_set(v)`: create a singleton set,
- `find_set(v)`: return the representative,
- `union_sets(a,b)`: merge two sets.

Disjoint Set Union (DSU)

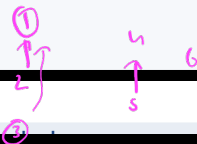
+

→ {1, 2, 3, 4, 5, 6} →
→ {1, 2, 3} {4, 5} {6}
5 1 4 6

→ 1, 2 → 1 ✓
1, 4 → 1, 4 ✗

A partition of vertices into disjoint sets, with operations:

- ✓ make_set(v): create a singleton set, {7}
- ✓ find_set(v): return the representative, → 1
- ✓ union_sets(a,b): merge two sets. (2, 4) → {1, 2, 3, 4, 5} {6}



- ✓ Path compression speeds up future finds.
- ✓ Union by size/rank keeps trees shallow.

We only need a fast test for whether two endpoints belong to the same component.

DSU: standard implementation idea

Find

Follow parent pointers until the representative is reached.

DSU: standard implementation idea

Find

Follow parent pointers until the representative is reached.

Path Compression

After a find, make every visited node point directly to the root.

DSU: standard implementation idea

Find

Follow parent pointers until the representative is reached.

Union

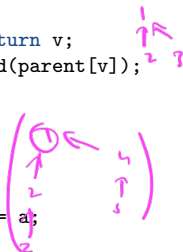
After a find, make every visited node point directly to the root.

Attach

Attach the smaller tree under the larger one.

```
int find(int v) {  
    if (parent[v] == v) return v;  
    return parent[v] = find(parent[v]);  
}
```

```
void unite(int a, int b) {  
    a = find(a);  
    b = find(b);  
    if (a != b) parent[b] = a;  
}
```



Kruskal with DSU: standard template

```
struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

sort(edges.begin(), edges.end());
for (auto e : edges) {
    if (find_set(e.u) != find_set(e.v)) {
        mst.push_back(e);
        cost += e.w;
        union_sets(e.u, e.v);
    }
}
```


Kruskal with DSU: standard template

```
struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

sort(edges.begin(), edges.end());
for (auto e : edges) {
    if (find_set(e.u) != find_set(e.v)) {
        mst.push_back(e);
        cost += e.w;
        union_sets(e.u, e.v);
    }
}
```

Sorting takes $O(m \log m)$, and DSU operations are almost constant amortized time.

Prim vs Kruskal



Aspect	Prim	Kruskal
Growth style	✓ Expands one connected tree	Merges many small components
Choice rule	Best edge leaving the current tree	Globally lightest edge that does not create a cycle
Main data structure	Priority queue / key array	DSU / Union-Find
Best for	Dense graphs or adjacency-based code	Sparse graphs and edge-list input
Complexity	$O(n^2)$ dense, $O(m \log n)$ sparse	$O(m \log m)$ with DSU

Topological sorting: definition

Given a directed graph, find an ordering of vertices such that every edge goes from an earlier vertex to a later vertex.

Topological sorting: definition

Given a directed graph, find an ordering of vertices such that every edge goes from an earlier vertex to a later vertex.

- A topological order may not be unique.
- A topological order exists if and only if the graph has no directed cycle.

Topological sorting: definition



✓ 1 4 2 3

Given a directed graph, find an ordering of vertices such that every edge goes from an earlier vertex to a later vertex.

- A topological order may not be unique.
- ✓ • A topological order exists if and only if the graph has no directed cycle.

Task scheduling, prerequisite dependencies, build systems, course planning.

Topological sort: intuition

- Every directed edge represents a dependency.
- If $u \rightarrow v$, then u must appear before v .
- So we are searching for a linear order that respects all edges.

Topological sort: intuition

- Every directed edge represents a dependency.
- If $u \rightarrow v$, then u must appear before v .
- So we are searching for a linear order that respects all edges.

Cycles make topological ordering impossible.



Topological sort by DFS finishing times

Run DFS. When a vertex finishes, append it to a list. The reverse of this list is a topological order.

Topological sort by DFS finishing times

Run DFS. When a vertex finishes, append it to a list. The reverse of this list is a topological order.

- If u is reachable from v , then DFS finishes u before v .
- So u appears earlier in the finishing list.
- Reversing the list places every edge forward.

```
void dfs(int v) {  
    used[v] = true;  
    for (int to : adj[v])  
        if (!used[to]) dfs(to);  
    order.push_back(v);  
}
```

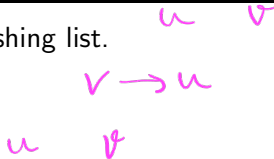
Why DFS order works

When $\text{dfs}(v)$ ends, every vertex reachable from v has already been completely processed. So v must come after all of them in the finishing list.

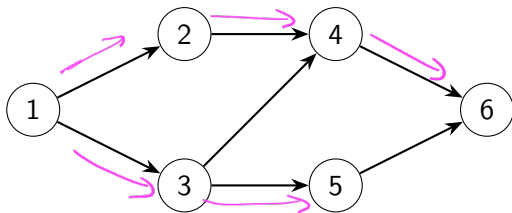
Why DFS order works

When $\text{dfs}(v)$ ends, every vertex reachable from v has already been completely processed. So v must come after all of them in the finishing list.

- For every directed edge $v \rightarrow u$, we get u before v in the finishing list.
- Reversing the list gives v before u in the final answer.
- Hence all edges point forward in the topological order.

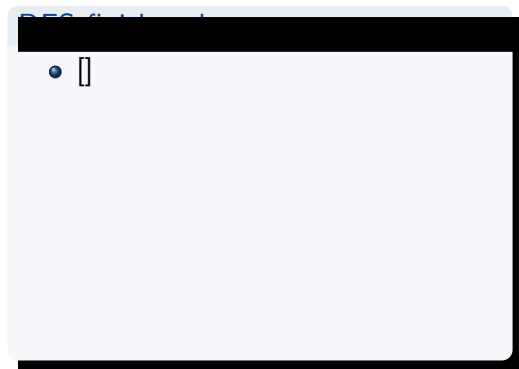


Topological sort: DFS example

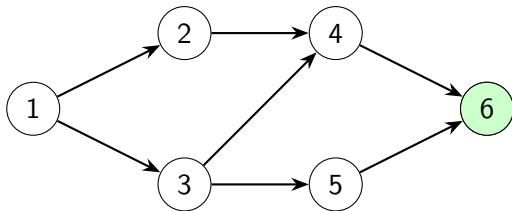


6 4 2 5 3 1

↳ 1 3 5 2 4 6 ✓



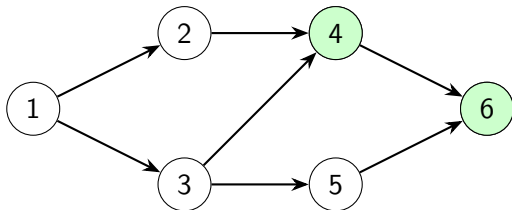
Topological sort: DFS example



- []
- [6]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

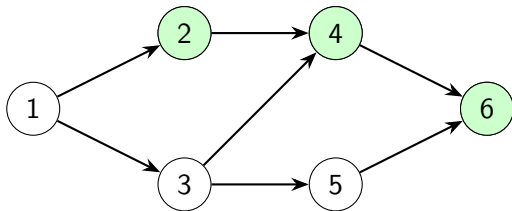
Topological sort: DFS example



- []
- [6]
- [6, 4]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

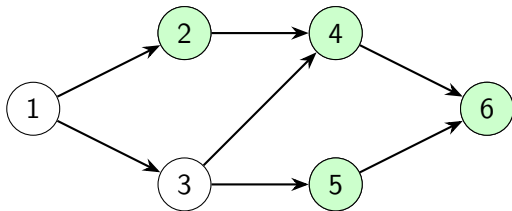
Topological sort: DFS example



- DFS finish order
- []
 - [6]
 - [6, 4]
 - [6, 4, 2]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

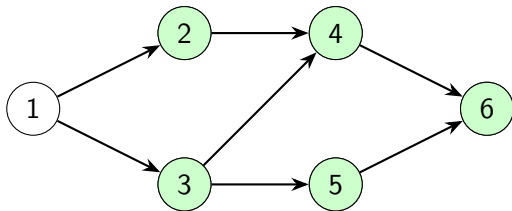
Topological sort: DFS example



- []
- [6]
- [6, 4]
- [6, 4, 2]
- [6, 4, 2, 5]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

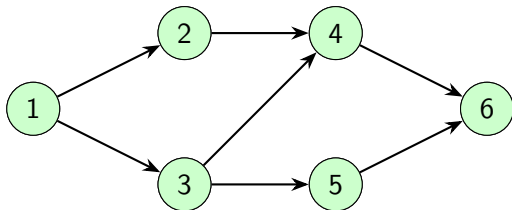
Topological sort: DFS example



- []
- [6]
- [6, 4]
- [6, 4, 2]
- [6, 4, 2, 5]
- [6, 4, 2, 5, 3]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

Topological sort: DFS example



- []
- [6]
- [6, 4]
- [6, 4, 2]
- [6, 4, 2, 5]
- [6, 4, 2, 5, 3]
- [6, 4, 2, 5, 3, 1]

- Reverse the finish order to get a topo order.
- One valid answer is 1, 3, 5, 2, 4, 6.

Complexities and takeaways

Complexities

- Prim: $O(n^2)$ in dense form, $O(m \log n)$ with a heap.
- Kruskal: sort edges + DSU, typically $O(m \log m)$.
- Topological sort by DFS: $O(n + m)$.

Summary

- MST is a greedy optimization problem on weighted undirected graphs.
- Prim grows one tree by repeatedly choosing the cheapest boundary edge.
- Kruskal sorts edges and uses DSU to avoid cycles.
- Topological sort gives a valid dependency order in a DAG.
- DFS finishing times provide a simple $O(n + m)$ method for topo sort.

